

Quantum Tunneling Through a Potential Barrier

Quantum tunneling is one of the most striking manifestations of quantum mechanics with no classical analog. In classical physics, a particle cannot pass through a potential barrier if its energy is less than the barrier height. However, in quantum mechanics, there is a non-zero probability for particles to “tunnel” through barriers that would be insurmountable according to classical physics.

This phenomenon has numerous practical applications: - Scanning tunneling microscopy (STM) - Nuclear fusion in stars - Alpha decay in radioactive nuclei - Tunnel diodes and other quantum electronic devices

In this lecture, we’ll explore quantum tunneling by numerically solving the time-dependent Schrödinger equation for a wavepacket approaching a potential barrier. We’ll implement two different numerical methods:

1. The **Crank-Nicolson method**: A finite difference approach that is unconditionally stable
2. The **Split-Step Fourier method**: An efficient approach that leverages fast Fourier transforms

```
#| edit: false
#| echo: false
#| execute: true
import numpy as np
import matplotlib.pyplot as plt
from scipy.constants import *
from scipy.sparse import diags, eye
from scipy.sparse.linalg import eigsh, inv
from scipy.fftpack import fft, ifft
from time import sleep, time

# Physical constants (using natural units for simplicity)
```

```

hbar = 1.0 # Reduced Planck constant in natural units
m = 1.0    # Particle mass in natural units

# Default plotting parameters
plt.rcParams.update({
    'font.size': 12,
    'lines.linewidth': 1,
    'lines.markersize': 5,
    'axes.labelsize': 11,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'xtick.top': True,
    'xtick.direction': 'in',
    'ytick.right': True,
    'ytick.direction': 'in',
})

def get_size(w, h):
    return (w/2.54, h/2.54)

```

The Time-Dependent Schrödinger Equation

The time-dependent Schrödinger equation governs the evolution of quantum states:

$$i\hbar \frac{\partial \Psi(x, t)}{\partial t} = \left(\frac{-\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x, t) \right) \Psi(x, t) \quad (1)$$

Where: - $\Psi(x, t)$ is the wavefunction - $\hbar$ is the reduced Planck constant - m is the particle mass - $V(x, t)$ is the potential energy function

This equation can be represented in both position space (as above) and momentum space. The transformation between these representations is accomplished using the Fourier transform.

Position and Momentum Space Representations

The wavefunction in momentum space, $\tilde{\Psi}(k, t)$, is related to the position-space wavefunction through the Fourier transform:

$$\tilde{\Psi}(k, t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \Psi(x, t) e^{-ikx} dx \quad (2)$$

And the inverse transformation:

$$\Psi(x, t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \tilde{\Psi}(k, t) e^{ikx} dk \quad (3)$$

The Schrödinger equation in momentum space becomes:

$$i\hbar \frac{\partial \tilde{\Psi}}{\partial t} = \frac{\hbar^2 k^2}{2m} \tilde{\Psi} + V\left(i \frac{\partial}{\partial k}\right) \tilde{\Psi} \quad (4)$$

This duality between position and momentum space will be particularly useful in our Split-Step method.

Method 1: Crank-Nicolson Scheme

The Crank-Nicolson method is a finite difference approach that offers second-order accuracy in time and is unconditionally stable. It effectively balances the forward and backward Euler methods.

Discretization of the Schrödinger Equation

Starting from the time-dependent Schrödinger equation, we can approximate the time derivative using centered differences:

$$\frac{\Psi(x, t + \Delta t) - \Psi(x, t)}{\Delta t} \approx -\frac{i}{\hbar} \left[\frac{1}{2} H \Psi(x, t) + \frac{1}{2} H \Psi(x, t + \Delta t) \right]$$

Where $H = \frac{-\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x)$ is the Hamiltonian operator.

Rearranging this equation:

$$\left(1 - \frac{\Delta t}{2i\hbar} H\right) \Psi(x, t + \Delta t) = \left(1 + \frac{\Delta t}{2i\hbar} H\right) \Psi(x, t)$$

And solving for $\Psi(x, t + \Delta t)$:

$$\Psi(x, t + \Delta t) = \left(1 - \frac{\Delta t}{2i\hbar} H\right)^{-1} \left(1 + \frac{\Delta t}{2i\hbar} H\right) \Psi(x, t)$$

This gives us a formula for propagating the wavefunction forward in time.

Setting Up the Simulation Domain

First, we define our spatial domain and time step:

```
#| autorun: false
# Set up the spatial grid
N = 500 # Number of grid points
x, dx = np.linspace(-100, 100, N, retstep=True) # Domain from -100 to 100

# Set up the time step
dt = 0.5 # Time step
```

Initial Wavepacket

We create a Gaussian wavepacket with an initial momentum (represented by k_0) directed toward the barrier:

```
#| autorun: false
def gauss_x(x, sigma, x0, k0):
    """
    Generate a Gaussian wavepacket.

    Parameters:
        x: Position array
        sigma: Width of the Gaussian
        x0: Center position
        k0: Momentum (wavenumber)

    Returns:
        Gaussian wavepacket
    """
    return (np.exp(-0.5 * ((x - x0)/sigma)**2 + 1j * x * k0)/(sigma * np.sqrt(np.pi)))

# Create initial wavepacket
k0 = 1.3 # Initial momentum
x0 = -50 # Initial position
sigma0 = 5 # Width of wavepacket

# Generate and normalize wavepacket
psi = gauss_x(x, sigma0, x0, k0)
prob = np.abs(psi)**2
```

```

psi = psi/np.sqrt(np.sum(prob)*dx) # Normalize

# Visualize initial wavepacket
plt.figure(figsize=get_size(10, 5))
plt.plot(x, np.abs(psi)**2, label='Initial wavepacket')
plt.xlabel('Position (x)')
plt.ylabel('Probability density  $|\Psi(x)|^2$ ')

plt.tight_layout()
plt.show()

```

Potential Barrier Setup

We create a potential barrier that the wavepacket will encounter:

```

#| autorun: false
# Define the potential barrier
barrier_width = 5
barrier_height = 1.0

# Create potential energy landscape
potential = np.array([barrier_height if 0.0 < x < barrier_width else 0.0 for x in x])

# Visualize the potential
plt.figure(figsize=get_size(10, 5))
plt.plot(x, potential, 'k', label='Potential barrier')
plt.xlabel('Position (x)')
plt.ylabel('Potential energy V(x)')

plt.tight_layout()
plt.show()

```

Constructing the Hamiltonian Matrix

We represent the Hamiltonian operator as a tridiagonal matrix:

```

#| autorun: false
from scipy import sparse

```

```
# Create the Hamiltonian matrix
h_diag = np.ones(N) / dx**2 + potential # Diagonal elements
h_non_diag = np.ones(N - 1) * (-0.5 / dx**2) # Off-diagonal elements

# Construct tridiagonal matrix
hamiltonian = sparse.diags([h_diag, h_non_diag, h_non_diag], [0, 1, -1])
```

Time Evolution Matrices

Following the Crank-Nicolson scheme, we create the matrices needed for time evolution:

```
#| autorun: false
from scipy.sparse import linalg as ln

# Construct the implicit and explicit operators
implicit = (sparse.eye(N) - dt / 2.0j * hamiltonian).tocsc()
explicit = (sparse.eye(N) + dt / 2.0j * hamiltonian).tocsc()

# Create the evolution matrix
evolution_matrix = ln.inv(implicit).dot(explicit).tocsr()
```

Animation Setup

Now we'll prepare for animating the wavepacket's evolution:

```
#| autorun: false
# Setup for animation
prob = np.abs(psi)**2
fig, ax = plt.subplots(1, 1, figsize=get_size(10, 6))
plt.xlim(-100, 100)
plt.ylim(0, 0.1)
plt.xlabel('Position (x)')
plt.ylabel('Probability density  $|\Psi(x)|^2$ ')

plt.plot(x, potential, 'k', label='Potential')

plt.tight_layout()
plt.draw()

background = fig.canvas.copy_from_bbox(ax.bbox)
```

```
points = ax.plot(x, prob, 'g', label='Wavepacket')[0]
plt.close()
```

Animation Execution

The animation will show the wavepacket approaching, partially reflecting from, and partially tunneling through the barrier:

```
# Setup the canvas for animation
canvas = Canvas(width=800, height=300, sync_image_data=False)
display(canvas)

# Run the animation
for i in range(200):
    # Evolve the wavefunction
    psi = evolution_matrix.dot(psi)
    prob = np.abs(psi)**2

    # Normalize
    norm = np.sum(prob)*dx
    prob /= norm
    psi /= np.sqrt(norm)

    # Update the plot
    fig.canvas.restore_region(background)
    ax.draw_artist(points)
    points.set_data(x, prob)

    fig.canvas.blit(ax.bbox)
    X = np.array(fig.canvas.renderer.buffer_rgba())

    with hold_canvas(canvas):
        canvas.clear()
        canvas.put_image_data(X)
        sleep(0.02)
```

Method 2: Split-Step Fourier Method

The Split-Step Fourier method leverages the efficiency of Fast Fourier Transforms (FFT) to solve the Schrödinger equation. It works by separately handling the kinetic and potential energy operators in their respective spaces where they are diagonal.

Mathematical Foundation

We can decompose the Hamiltonian into two parts:

$$\hat{H} = \hat{T} + \hat{V} \quad (5)$$

Where:

- $\hat{T} = \frac{-\hbar^2}{2m} \frac{\partial^2}{\partial x^2}$ is the kinetic energy operator
- $\hat{V} = V(x)$ is the potential energy operator

The formal solution to the Schrödinger equation is:

$$\Psi(x, t + \Delta t) = e^{-i\hat{H}\Delta t/\hbar} \Psi(x, t) \quad (6)$$

Using the Trotter-Suzuki approximation, we can split this exponential:

$$e^{-i(\hat{T}+\hat{V})\Delta t/\hbar} \approx e^{-i\hat{V}\Delta t/\hbar} e^{-i\hat{T}\Delta t/\hbar} + \mathcal{O}(\Delta t^2) \quad (7)$$

i The Trotter-Suzuki Approximation

The Trotter-Suzuki approximation is a fundamental technique in quantum physics that allows us to handle non-commuting operators in exponential form. When we have a Hamiltonian that is the sum of two operators $\hat{H} = \hat{A} + \hat{B}$ that don't commute ($[\hat{A}, \hat{B}] \neq 0$), we cannot simply write $e^{-i(\hat{A}+\hat{B})t} = e^{-i\hat{A}t} e^{-i\hat{B}t}$.

Instead, the first-order Trotter formula gives us:

$$e^{-i(\hat{A}+\hat{B})\Delta t} = e^{-i\hat{A}\Delta t} e^{-i\hat{B}\Delta t} + \mathcal{O}(\Delta t^2)$$

For better accuracy, we can use the second-order symmetric splitting:

$$e^{-i(\hat{A}+\hat{B})\Delta t} = e^{-i\hat{A}\Delta t/2} e^{-i\hat{B}\Delta t} e^{-i\hat{A}\Delta t/2} + \mathcal{O}(\Delta t^3)$$

In the context of the Schrödinger equation, this allows us to separately handle the kinetic energy operator ($\hat{T}$) and potential energy operator ($\hat{V}$) in their respective spaces where they are diagonal, making numerical solutions much more efficient.

This gives us:

$$\Psi(x, t + \Delta t) \approx e^{-i\hat{V}\Delta t/\hbar} e^{-i\hat{T}\Delta t/\hbar} \Psi(x, t) \quad (8)$$

The key insight is that $\hat{T}$ is diagonal in momentum space, while $\hat{V}$ is diagonal in position space. This leads to the algorithm:

1. Apply the potential evolution operator: $\Psi_1(x) = e^{-i\hat{V}\Delta t/\hbar}\Psi(x, t)$
2. Transform to momentum space: $\tilde{\Psi}_1(k) = \mathcal{F}[\Psi_1(x)]$
3. Apply the kinetic evolution operator: $\tilde{\Psi}_2(k) = e^{-i\hat{T}\Delta t/\hbar}\tilde{\Psi}_1(k)$
4. Transform back to position space: $\Psi(x, t + \Delta t) = \mathcal{F}^{-1}[\tilde{\Psi}_2(k)]$

Where $\mathcal{F}$ represents the Fourier transform.

i Diagonal Operators in Position and Momentum Space

In quantum mechanics, an operator is diagonal in a particular basis if it acts on states in that basis by simply multiplying them by a scalar value.

Potential Energy Operator in Position Space: The potential energy operator $\hat{V}$ is defined by its action on a wavefunction:

$$\hat{V}\Psi(x) = V(x)\Psi(x)$$

This is a simple multiplication operation in position space. If we represent the wavefunction as a vector of values at different positions, the potential operator becomes a diagonal matrix with $V(x_i)$ as its diagonal elements. This is why we can apply the potential evolution operator directly by multiplication:

$$e^{-i\hat{V}\Delta t/\hbar}\Psi(x) = e^{-iV(x)\Delta t/\hbar}\Psi(x)$$

Kinetic Energy Operator in Momentum Space: The kinetic energy operator in position space is a differential operator:

$$\hat{T} = \frac{-\hbar^2}{2m} \frac{\partial^2}{\partial x^2}$$

This is not diagonal in position space (it's a second derivative). However, in momentum space, it takes a simple form:

$$\hat{T}\tilde{\Psi}(k) = \frac{\hbar^2 k^2}{2m} \tilde{\Psi}(k)$$

In momentum space, the kinetic energy operator simply multiplies each momentum component by $\frac{\hbar^2 k^2}{2m}$, making it diagonal. This is why we can efficiently apply the kinetic evolution operator in momentum space:

$$e^{-i\hat{T}\Delta t/\hbar}\tilde{\Psi}(k) = e^{-i\hbar k^2 \Delta t/(2m)} \tilde{\Psi}(k)$$

The Split-Step method exploits this property by applying each operator in the space where it's diagonal, using Fourier transforms to switch between spaces.

Setting Up the Simulation Domain

```
#| autorun: false
# Set natural units
hbar = 1
m = 1

# Set up spatial domain
N = 2**11 # Number of grid points (power of 2 for efficient FFT)
dx = 0.1 # Spatial step
x = dx * (np.arange(N) - 0.5 * N) # Center the grid at x=0

# Set up time step
dt = 0.005 # Time step
```

Potential Barrier Setup

```
#| autorun: false
# Define barrier parameters
V0 = 1.0 # Barrier height
a = 5.0 # Barrier width

# Create potential energy landscape
V_x = np.zeros_like(x)
V_x[(x > 0) & (x < a)] = V0 # Central barrier
V_x[np.abs(x) > 98] = 1e6 # High barriers at domain edges to prevent boundary effects

# Visualize the potential
plt.figure(figsize=get_size(10, 6))
plt.plot(x, V_x, 'k', label='Potential')
plt.ylim(0, 2)
plt.xlabel('Position (x)')
plt.ylabel('Potential energy V(x)')

plt.tight_layout()
plt.show()
```

Initial Wavepacket

```
#| autorun: false
# Create initial wavepacket
k0 = 1.3      # Initial momentum
x0 = -50      # Initial position
sigma0 = 5    # Width of wavepacket

# Generate and normalize wavepacket
psi = gauss_x(x, sigma0, x0, k0)
prob = np.abs(psi)**2
dx = x[1] - x[0]
psi = psi/np.sqrt(np.sum(prob)*dx) # Normalize

# Visualize initial wavepacket
plt.figure(figsize=get_size(10, 6))
plt.plot(x, np.abs(psi)**2, label='Initial wavepacket')
plt.plot(x, V_x, 'k', label='Potential')
plt.xlabel('Position (x)')
plt.ylabel(r'Probability density  $|\Psi(x)|^2$ ')

plt.tight_layout()
plt.show()
```

Setting Up the Fourier Transform Space

```
#| autorun: false
# Set up momentum space grid
N = len(x)
dx = x[1] - x[0]
k0 = -np.pi/dx # Nyquist frequency
dk = 2*np.pi / (N * dx)
k = k0 + dk * np.arange(N)

# Prepare wavefunction for FFT
psi_modx = psi * np.exp(-1j * k[0] * x) * dx / np.sqrt(2 * np.pi)
```

Precomputing Phase Factors

Both the potential and kinetic evolution operators involve phase factors that remain constant throughout the simulation. We can precompute them:

```
#| autorun: false
# Precompute phase factors for potential and kinetic terms
phase_x = np.exp(-1j * V_x * dt / hbar) # Potential evolution
phase_k = np.exp(-1j * hbar * k**2 * dt / (2 * m)) # Kinetic evolution
```

Animation Setup

```
#| autorun: false
# Setup for animation
fig, ax = plt.subplots(1, 1, figsize=get_size(10, 6))
plt.xlim(-100, 100)
plt.ylim(0, 0.1)
plt.xlabel('Position (x)')
plt.ylabel('Probability density  $|\Psi(x)|^2$ ')

plt.plot(x, V_x, 'k', label='Potential')

plt.tight_layout()
plt.draw()

background = fig.canvas.copy_from_bbox(ax.bbox)
points = ax.plot(x, np.abs(psi)**2, 'g')[0]
plt.close()
```

Animation Execution

The animation will show the wavepacket tunneling through the potential barrier:

```
# Setup the canvas for animation
canvas = Canvas(width=800, height=300, sync_image_data=False)
display(canvas)

# Run the animation
for i in range(100):
```

```

fig.canvas.restore_region(background)
ax.draw_artist(points)

# Perform multiple time steps between animation frames for smoother results
for j in range(10):
    # Apply potential evolution
    psi_modx = psi_modx * phase_x

    # Transform to momentum space
    psi_modk = fft(psi_modx)

    # Apply kinetic evolution
    psi_modk = psi_modk * phase_k

    # Transform back to position space
    psi_modx = ifft(psi_modk)

    # Convert back to standard wavefunction
    psi = psi_modx * np.exp(1j * k[0] * x) * np.sqrt(2 * np.pi) / dx

    # Normalize
    prob = np.abs(psi)**2
    norm = np.sum(prob) * dx
    psi = psi / np.sqrt(norm)
    prob = prob / norm

# Update the plot
points.set_data(x, prob)

fig.canvas.blit(ax.bbox)
X = np.array(fig.canvas.renderer.buffer_rgba())

with hold_canvas(canvas):
    canvas.clear()
    canvas.put_image_data(X)
    sleep(0.01)

```

Physical Interpretation and Analysis

What's Happening in the Simulation?

In both methods, we observe:

1. **Incident Wavepacket:** The Gaussian wavepacket approaches the barrier from the left
2. **Reflection and Transmission:** Upon reaching the barrier, the wavepacket splits:
 - A portion is reflected back (reflection)
 - A portion passes through the barrier (tunneling)
3. **Interference:** After reflection, the incident and reflected components interfere, creating oscillations in the probability density

Factors Affecting Tunneling Probability

The tunneling probability depends on several factors:

1. **Barrier Height (V_0):** Higher barriers reduce tunneling probability exponentially
2. **Barrier Width (a):** Wider barriers reduce tunneling probability exponentially
3. **Particle Energy:** Higher energy particles (larger k_0) have higher tunneling probabilities
4. **Particle Mass:** Heavier particles have lower tunneling probabilities

Connection to Quantum Theory

For a plane wave with energy $E = \frac{\hbar^2 k^2}{2m}$ incident on a rectangular barrier of height V_0 and width a , the transmission coefficient in the case $E < V_0$ is:

$$T \approx \frac{16E(V_0 - E)}{V_0^2} e^{-2\kappa a} \quad (9)$$

Where $\kappa = \sqrt{\frac{2m(V_0 - E)}{\hbar^2}}$ is the attenuation coefficient inside the barrier.

This exponential dependence on barrier width explains why quantum tunneling is primarily observed at microscopic scales.

Comparison of Numerical Methods

Both the Crank-Nicolson and Split-Step methods accurately simulate quantum tunneling, but they have different characteristics:

Crank-Nicolson Method

- **Pros:** Unconditionally stable, second-order accurate in both space and time
- **Cons:** Requires matrix inversion, which can be computationally expensive for large systems

Split-Step Fourier Method

- **Pros:** Very efficient for periodic boundary conditions, leverages fast FFT algorithms
- **Cons:** Requires care with boundary conditions, first-order accurate in time (though higher-order splitting schemes exist)

Further Explorations

Here are some suggestions for further exploration:

1. **Momentum Space Analysis:** Examine how the wavepacket's momentum distribution changes during tunneling
2. **Barrier Parameters:** Investigate how tunneling probability changes with barrier height and width
3. **Wavepacket Parameters:** Explore how the initial momentum and width of the wavepacket affect tunneling
4. **Different Potentials:** Try other potential shapes, such as:
 - Double barriers (resonant tunneling)
 - Smooth potentials (e.g., Gaussian barriers)
 - Time-dependent barriers
5. **Quantum Reflection:** Study reflection from potential wells instead of barriers

Implementation Exercises

1. Modify the code to calculate and plot the transmission and reflection coefficients as a function of incident momentum
2. Implement a double-barrier potential and observe resonant tunneling
3. Visualize the wavefunction in momentum space alongside position space
4. Implement a higher-order Split-Step method (e.g., second-order Strang splitting)
5. Add measurement simulation to observe wavefunction collapse